

기념탑

사마르칸드에 있는 Registan 광장은 N 개의 기념탑이 광장 중심을 지나는 1차원 직선 위에 놓여 있다. 기념탑은 0부터 $N - 1$ 까지 번호가 매겨져 있다. 기념탑 i 는 ($0 \leq i < N$) 처음에 정수 좌표 $X[i]$ 에 놓여 있다. 광장 중심은 좌표 0이다.

건축가들은 중심에 대해서 완벽한 대칭을 이룰 것을 요구한다. 몇 개의 기념탑을 옮겨서 최종적으로 좌표 0에 대해 대칭이 되게 하려고 한다. 이미 대칭인 경우에는 0개를 옮겨도 된다. 엄밀하게는 다음과 같다.

- 모든 기념탑은 정수 좌표에 놓여야 한다.
- 각 정수 좌표에는 **0, 1, 또는 여러 개의** 기념탑이 놓일 수 있다.
- $x > 0$ 인 모든 정수에 대해서, 좌표 x 에 놓인 기념탑의 수는 좌표 $-x$ 에 놓인 기념탑의 수와 같아야 한다. 좌표 0에 놓을 수 있는 기념탑의 수에는 제약이 없다. (0개를 놓아도 된다.)

그러나, M 개의 기념탑은 오래되어서 옮길 수 없다. 옮길 수 없는 기념탑들의 번호는 $P[0], P[1], \dots, P[M - 1]$ 이다. 이 M 개의 기념탑들은 원래 위치에 있어야 한다. 나머지 $N - M$ 개의 기념탑은 어떤 정수 좌표로도 옮길 수 있다.

좌표 a 에 있는 기념탑 하나를 좌표 b 로 옮기는데 드는 비용은 $|a - b|$ 이다. 전체 비용은 각 기념탑을 옮기는데 드는 비용의 총합이다. 한 기념탑을 옮길때 중간에 만나는 다른 기념탑들로부터 방해받지 않는다.

당신이 할 일은 좌표 0에 대칭이 되게 기념탑들을 옮기는데 드는 비용의 최소값을 구하거나, 주어진 제약 조건에서는 이렇게 할 수 없다는 것을 알아내는 것이다.

Implementation Details

다음 함수를 구현해야 한다.

```
long long get_cost(std::vector<int> X, std::vector<int> P)
```

- X : 길이 N 인 오름차순으로 정렬된 배열로 (중복이 있을 수 있다) 기념탑들의 처음 좌표를 나타낸다.
- P : 길이 M 인 오름차순으로 정렬된 배열로 오래되어 옮길 수 없는 기념탑들의 번호를 나타낸다.
- 이 함수는 각 테스트케이스에 대해 정확히 한 번 호출된다.

이 함수는 정수값을 리턴해야 한다. 만약 기념탑들이 대칭이 되게 옮길 수 있다면 최소 비용을 리턴하고, 만약 불가능하다면 -1 을 리턴한다.

Constraints

- $1 \leq N \leq 500\,000$
- $0 \leq M \leq N$
- $-10^9 \leq X[0] \leq X[1] \leq \dots \leq X[N-1] \leq 10^9$
- $0 \leq P[0] < P[1] < \dots < P[M-1] < N$

Subtasks

Subtask	Score	Additional Constraints
1	3	$M = N$
2	4	$M = 0$
3	5	$0 \leq i < M$ 인 i 에 대해 항상 $X[P[i]] < 0$.
4	6	$N \leq 10$
5	5	$N \leq 19$
6	5	$N \leq 32$
7	13	$N \leq 200$
8	17	$N \leq 4000$
9	13	$M \leq 4000$
10	29	추가적인 제약이 없다.

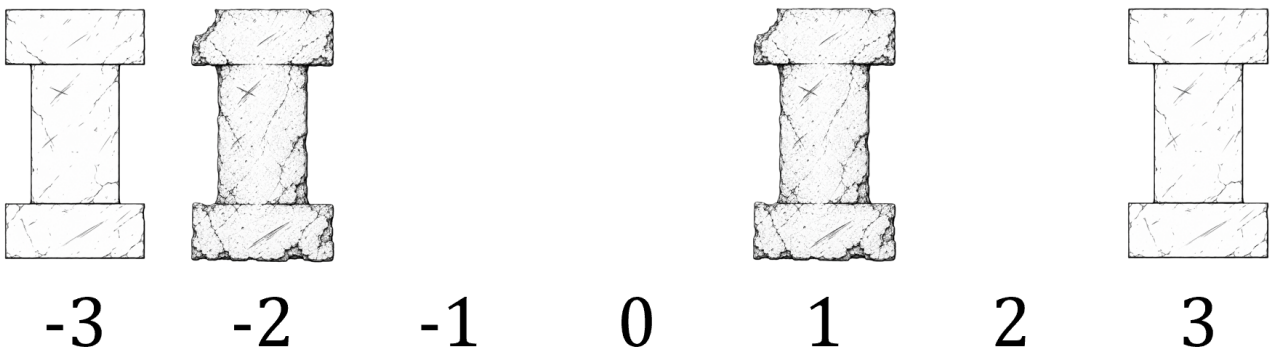
Examples

Example 1

다음 호출을 생각해보자.

```
get_cost([-3, -2, 1, 3], [1, 2])
```

기념탑들의 처음 배치는 다음 그림과 같다.

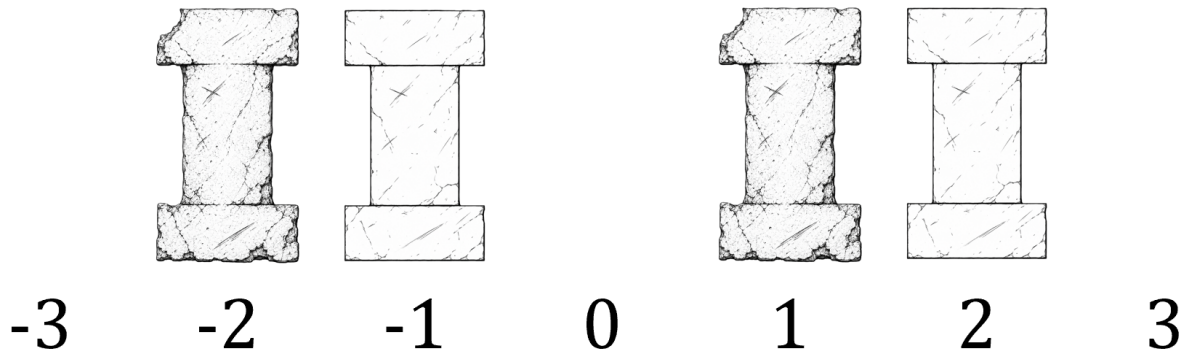


$N = 4$ 개의 기념탑이 있고, 처음에는 좌표 $[-3, -2, 1, 3]$ 에 있다. 오래된 기념탑의 번호는 $P[0] = 1$ 과 $P[1] = 2$ 이다. 즉, 좌표 -2 와 좌표 1 에 있는 기념탑은 움직일 수 없다. 나머지 좌표 -3 와 좌표 3 에 있는 기념탑은 움직일 수 있다.

최소 비용으로 대칭을 만들려면 다음과 같이 기념탑을 움직여야 한다.

- 좌표 -3 에 있는 기념탑을 좌표 -1 로 비용 $|(-3) - (-1)| = 2$ 로 옮긴다.
- 좌표 3 에 있는 기념탑을 좌표 2 로 비용 $|3 - 2| = 1$ 로 옮긴다.

이렇게 움직이면, 기념탑은 대칭으로 배치되게 된다.



전체 비용은 $2 + 1 = 3$ 이므로 함수는 3을 리턴해야 한다.

Example 2

다음 호출을 생각해보자.

```
get_cost([2, 2, 2, 3], [])
```

여기서 $M = 0$ 은 오래된 기념탑이 없다는 뜻이며, 모든 기념탑을 움직일 수 있다. 최소 비용으로 문제를 푸는 방법 중 하나는 모든 기념탑을 좌표 0으로 옮기는 것이다. 각각 움직이는데 드는 비용은 다음과 같다.

- 기념탑 0, 1, 2를 움직이는데 $|2 - 0| = 2$.
- 기념탑 3을 움직이는데 $|3 - 0| = 3$.

전체 비용은 $2 + 2 + 2 + 3 = 9$ 이므로 함수는 9를 리턴해야 한다. 비용의 합이 같은 다른 방법도 있다.

Example 3

다음 호출을 생각해보자.

```
get_cost([1, 2, 3, 4], [0, 1, 2, 3])
```

4개의 기념탑이 모두 오래되어 움직일 수 없다. 좌표가 모두 양수이기 때문에, 좌표 0에 대칭으로 기념탑을 옮기는 방법은 없다. 따라서, 이 함수는 -1 을 리턴해야 한다.

Sample Grader

Input Format:

```
N M
X[0] X[1] ... X[N-1]
P[0] P[1] ... P[M-1]
```

만약 M 이 0이면 세번째 줄은 빌 수 있다.

Output Format:

```
C
```

여기에서 C 는 `get_cost`가 리턴한 값이다.